

Name- Bhumika Shrotriya

Course Name – Data Analytics With GenAI

Subject – Data Processing & ETL

Topic – Data Loading

Q1. Data Understanding — All Data Quality Issues

Answer 1:

Sno	Issue Type	Affected Record(s)	Description
1	Duplicate Primary Key	O101 (rows 1 & 4)	Order_ID O101 appears twice with identical data
2	Missing / Null Value	O102	Sales_Amount is Null
3	Wrong Data Type	O104	Sales_Amount = "Three Thousand" (text instead of numeric)
4	Inconsistent Date Format	O103	Format 2024/01/18 differs from DD-MM-YYYY used elsewhere
5	Exact Duplicate Row	O101 (rows 1 & 4)	All four field values are completely identical

Q2. Primary Key Validation

Answer 2:

a) Is the dataset violating the Primary Key rule?

Yes. A Primary Key must be UNIQUE and NOT NULL. Order_ID = O101 appears in two rows, directly violating the uniqueness constraint.

b) Which record(s) cause this violation?

Row	Order_ID	Customer_ID	Sales_Amount	Order_Date
Row 1	O101	C001	4500	12-01-2024
Row 4	O101	C001	4500	12-01-2024

Both rows are completely identical. When loading into a SQL table with Order_ID as PRIMARY KEY, the database will throw a **PRIMARY KEY constraint violation error** and reject the second insert.

Detection in SQL:

```
SELECT Order_ID, COUNT(*) AS occurrence_count
FROM staging_orders
GROUP BY Order_ID
HAVING COUNT(*) > 1;
```

Output:

Order_ID	occurrence_count
O101	2

Q3. Missing Value Analysis

Answer 3:

Column containing missing values: Sales_Amount

a) Affected Records:

Order_ID	Customer_ID	Sales_Amount	Order_Date
O102	C002	Null	15-01-2024

Detection in SQL:

```
SELECT *
```

```
FROM staging_orders
```

```
WHERE Sales_Amount IS NULL;
```

b) Why loading records with missing values is risky:

1. Silent exclusion from aggregates: SQL's SUM(Sales_Amount) and AVG(Sales_Amount) automatically ignore NULLs. The Total Sales KPI on a dashboard will be understated with no error or warning shown.

2. NOT NULL constraint failure: If the target table defines Sales_Amount NOT NULL, the entire load batch may be rejected, blocking all records — not just the problematic one.

3. Downstream business logic breaks: Processes such as commission calculation, tax computation, and inventory valuation that depend on Sales_Amount will either error out or silently produce wrong results.

4. BI misinterpretation: Some BI tools treat NULL as zero, falsely suggesting a zero-revenue transaction rather than an unknown or missing value.

Q4. Data Type Validation

Answer 4:

a) Records that will fail numeric validation:

Order_ID	Sales_Amount	Reason
O102	Null	Missing value — not a valid numeric
O104	"Three Thousand"	Text string — cannot be cast to DECIMAL

Detection SQL:

```
SELECT *  
FROM staging_orders  
WHERE Sales_Amount IS NULL  
OR Sales_Amount NOT REGEXP '^[0-9]+(\.[0-9]+)?$';
```

b) What happens if loaded into a SQL table with Sales_Amount as DECIMAL:

```
CREATE TABLE Sales_Transactions (  
    Order_ID VARCHAR(10) PRIMARY KEY,  
    Customer_ID VARCHAR(10) NOT NULL,  
    Sales_Amount DECIMAL(10, 2),  
    Order_Date DATE  
);
```

- **O104** → SQL throws a hard error: Incorrect DECIMAL value: 'Three Thousand'. In strict SQL mode, the entire INSERT batch is aborted. In non-strict mode, the value is converted to 0 or NULL silently, causing revenue loss in reports.
- **O102** → If Sales_Amount is NOT NULL, this row also fails and gets rejected. If NULLs are permitted, it loads but contributes nothing to SUM-based KPIs.
- Either way, **real revenue of ₹3,000 (O104) and ₹unknown (O102) disappears from the target table** without any visible alert to business users.

Q5. Date Format Consistency

Answer 5:

a) All date formats present in the dataset:

Order_ID	Order_Date Value	Format Identified
O101	12-01-2024	DD-MM-YYYY
O102	15-01-2024	DD-MM-YYYY
O103	2024/01/18	YYYY/MM/DD
O101 (dup)	12-01-2024	DD-MM-YYYY
O104	20-01-2024	DD-MM-YYYY
O105	25-01-2024	DD-MM-YYYY

Two distinct formats present: DD-MM-YYYY and YYYY/MM/DD

b) Why this is a problem during data loading:

- 1. **SQL DATE columns require ISO 8601 format (YYYY-MM-DD).** Both formats in the dataset differ from this standard, so all rows need explicit conversion before loading.
- 2. **Ambiguous parsing leads to wrong dates.** The string 12-01-2024 could be read as January 12 or December 1 depending on database locale settings. The same data produces different date values in different environments.
- 3. **ETL transformation using a single format pattern will fail partially.** A function like STR_TO_DATE(Order_Date, '%d-%m-%Y') correctly handles DD-MM-YYYY rows but returns NULL for O103 (2024/01/18), silently breaking that record's date.

Normalization SQL:

-- Standardize all dates to YYYY-MM-DD

```
UPDATE staging_orders
SET Order_Date = STR_TO_DATE(Order_Date, '%d-%m-%Y')
WHERE Order_Date REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$';

UPDATE staging_orders
SET Order_Date = STR_TO_DATE(Order_Date, '%Y/%m/%d')
WHERE Order_Date REGEXP '^[0-9]{4}/[0-9]{2}/[0-9]{2}$';
```

Q6. Load Readiness Decision

Answer 6:

a) Should this dataset be loaded directly into the database?

No.

b) Justification:

Reason 1 : Primary Key Violation: O101 appears twice. Loading directly will either abort the entire batch (strict mode) or silently create a duplicate that corrupts order-level reporting and double-counts revenue.

Reason 2 : Invalid Data Type: O104 contains "Three Thousand" in a DECIMAL column. This causes a type casting error that can abort the full load job or silently insert 0, making ₹3,000 of real revenue disappear.

Reason 3 : Null in Critical Field: O102 has a Null Sales_Amount. Without a handling strategy, this silently distorts Total Sales, Average Order Value, and all revenue-based KPIs.

Reason 4 : Inconsistent Date Formats: Two different formats in Order_Date will cause incorrect or NULL date values after loading, breaking all time-series reports, month-over-month comparisons, and date-filtered queries.

Q7. Pre-Load Validation Checklist

Answer 7:

Perform the following SQL-based checks on the staging table before loading to the target:

Check 1 — Primary Key Uniqueness:

```
SELECT Order_ID, COUNT(*) AS cnt
FROM staging_orders
GROUP BY Order_ID
HAVING COUNT(*) > 1;
```

Check 2 — NULL / Missing Values in Critical Fields:

```
SELECT *
FROM staging_orders
WHERE Order_ID IS NULL
   OR Customer_ID IS NULL
   OR Sales_Amount IS NULL
   OR Order_Date IS NULL;
```

Check 3 — Data Type Validity for Sales_Amount:

```
SELECT *  
FROM staging_orders  
WHERE Sales_Amount NOT REGEXP '^[0-9]+(\.[0-9]+)?$';
```

Check 4 — Date Format Consistency:

```
SELECT *  
FROM staging_orders  
WHERE Order_Date NOT REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$'  
AND Order_Date NOT REGEXP '^[0-9]{4}/[0-9]{2}/[0-9]{2}$';
```

Check 5 — Full Row Duplicate Detection:

```
SELECT Order_ID, Customer_ID, Sales_Amount, Order_Date, COUNT(*) AS cnt  
FROM staging_orders  
GROUP BY Order_ID, Customer_ID, Sales_Amount, Order_Date  
HAVING COUNT(*) > 1;
```

Check 6 — Referential Integrity (Customer_ID exists in master):

```
SELECT s.*  
FROM staging_orders s  
LEFT JOIN Customers_Master c ON s.Customer_ID = c.Customer_ID  
WHERE c.Customer_ID IS NULL;
```

Check 7 — Business Rule: Sales_Amount must be > 0:

```
SELECT *  
FROM staging_orders  
WHERE CAST(Sales_Amount AS DECIMAL) <= 0;
```

Check 8 — Row Count Reconciliation:

```
-- Compare source count vs staging count  
SELECT COUNT(*) AS staging_row_count FROM staging_orders;  
  
-- Manually verify this matches the expected source extract count
```

Q8. Cleaning Strategy — Step by Step

Answer 8:

Step 1: Remove Exact Duplicate Rows

Keep only one instance of O101 and delete the duplicate:

```
DELETE FROM staging_orders
WHERE Order_ID = 'O101'
AND ROWID NOT IN (
    SELECT MIN(ROWID)
    FROM staging_orders
    WHERE Order_ID = 'O101'
);
```

Step 2: Quarantine Records with NULL Sales_Amount

Move O102 to a rejected records table for manual review:

```
INSERT INTO rejected_records (Order_ID, Customer_ID, Sales_Amount, Order_Date, rejection_reason,
rejected_at)
SELECT Order_ID, Customer_ID, Sales_Amount, Order_Date, 'NULL Sales_Amount', NOW()
FROM staging_orders
WHERE Sales_Amount IS NULL;
```

```
DELETE FROM staging_orders WHERE Sales_Amount IS NULL;
```

Step 3: Fix Invalid Data Type in Sales_Amount (O104)

Map "Three Thousand" to its numeric value 3000:

```
UPDATE staging_orders
SET Sales_Amount = '3000'
WHERE Sales_Amount = 'Three Thousand';
```

If the text value cannot be confidently mapped, quarantine it as well using the same pattern as Step 2.

Step 4: Standardize All Date Formats to YYYY-MM-DD

-- Convert DD-MM-YYYY format

```
UPDATE staging_orders
```



```
SET Order_Date = CONCAT(  
    SUBSTRING(Order_Date, 7, 4), '-',  
    SUBSTRING(Order_Date, 4, 2), '-',  
    SUBSTRING(Order_Date, 1, 2)  
)  
  
WHERE Order_Date REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$';
```

-- Convert YYYY/MM/DD format

```
UPDATE staging_orders  
  
SET Order_Date = REPLACE(Order_Date, '/', '-')  
  
WHERE Order_Date REGEXP '^[0-9]{4}/[0-9]{2}/[0-9]{2}$';
```

Step 5: Re-run Full Validation Checklist

Execute all checks from Q7 again to confirm zero issues remain before proceeding to load.

Step 6: Load Clean Records to Target Table

```
INSERT INTO Sales_Transactions (Order_ID, Customer_ID, Sales_Amount, Order_Date)  
  
SELECT Order_ID, Customer_ID, CAST(Sales_Amount AS DECIMAL(10,2)), CAST(Order_Date AS DATE)  
  
FROM staging_orders;
```

Q9. Loading Strategy Selection

Answer 9:

a) Full Load or Incremental Load?

Incremental Load (Delta Load)

b) Justification:

1. Daily data means only new records need to be added. Since this represents daily sales, orders from previous days already exist in the warehouse. Re-loading all historical data every day is unnecessary and wasteful.

2. Performance and scalability. As the sales table grows over months and years, a Full Load would become progressively slower and more resource-intensive. Incremental Load processes only the day's new records — a fraction of the total volume.

3. Preserves historical data integrity. Full Load typically truncates and reloads the target table, meaning any corrected historical records or manually added annotations would be destroyed on each run.

4. Supports change tracking. Using Order_Date as a watermark, each daily run extracts only records where Order_Date = CURRENT_DATE - 1, ensuring no overlap or gap with previous loads.

Implementation SQL:

```
INSERT INTO Sales_Transactions (Order_ID, Customer_ID, Sales_Amount, Order_Date)
SELECT Order_ID, Customer_ID, CAST(Sales_Amount AS DECIMAL(10,2)), CAST(Order_Date AS DATE)
FROM staging_orders
WHERE Order_Date = CURRENT_DATE - INTERVAL 1 DAY;
```

Q10. BI Impact Scenario

Answer 10:

a) Incorrect results in Total Sales KPI:

Order_ID	Sales_Amount	Counted in SUM?	Issue
O101	4500	Yes	-
O102	Null	Silently excluded	Missing ₹unknown
O103	3200	Yes	-
O101 (dup)	4500	Yes	Double counted
O104	"Three Thousand"	Cast error / 0	Missing ₹3,000
O105	5100	Yes	-

Dashboard shows: 4500 + 3200 + 4500 + 5100 = ₹17,300

Actual correct Total Sales: 4500 + 3200 + 3000 + 5100 = ₹15,800

The dashboard **overstates revenue by ₹1,500** due to the duplicate O101, and **understates it** by excluding O104's ₹3,000 — two errors that partially mask each other, making the problem even harder to detect manually.

b) Records that would cause misleading insights:

Record	Misleading Effect
O101 (duplicate)	Revenue and order count both inflated — appears as two separate orders
O102 (Null amount)	Order appears in record count but contributes ₹0 — Average Order Value is artificially lowered

Record	Misleading Effect
O104 ("Three Thousand")	₹3,000 of real revenue completely missing from Total Sales and all related metrics
O103 (wrong date format)	If date is parsed incorrectly, this order appears in the wrong month — distorts monthly trend charts

c) Why BI tools would NOT detect these issues automatically:

- 1. BI tools assume the data layer is already clean.** Tools like Power BI, Tableau, and Looker are designed for aggregation and visualization, not data quality auditing. They trust whatever the database serves them.
- 2. NULL exclusion is silent.** When SUM(Sales_Amount) skips NULL rows, no warning, flag, or asterisk appears on the dashboard. The number simply reflects fewer records with no indication of what was omitted.
- 3. Duplicates appear as legitimate separate records.** O101 appearing twice looks like two valid orders to a BI tool. Without a deduplication rule explicitly coded into the data model or report, the tool has no basis to question it.
- 4. Type coercion varies by tool.** Some BI tools may silently convert "Three Thousand" to 0 when computing numeric aggregates, making ₹3,000 of revenue disappear without any visible error in the dashboard or data model.
- 5. No semantic / business rule awareness.** BI tools do not know that Order_ID must be unique, or that Sales_Amount must always be a positive number. These rules must be enforced upstream in the ETL pipeline — BI tools only report what they receive.